SABI ENGINEERING & CONTRACTING LLC · DUBAI, UAE

drawtoboq.com

System Workflow & Architecture

Automated MEP estimation pipeline from RFQ email to client-ready BOQ

Phase 1	Phase 2	Phase 3	Phase 4
Information Sufficiency	Bid Decision	Quantities	Final Quote

15	5	14	12
MAIN steps	Confirmation gates	Electrical sub-steps	BOQ sections

Built on

Next.js 14 · React 18 · TypeScript · Tailwind · Supabase (Postgres + S3) · Vercel · Google Gemini · Anthropic Claude Sonnet 4.6 (via Nexaproc gateway) · pdfkit · ExcelJS · dxf-parser · pdf-parse · tesseract.js · OpenClaw CLI (WhatsApp) · Gmail API

What this system does

drawtoboq.com is an automated estimation pipeline for SABI, an MEP contractor in Dubai. A new RFQ email lands in estimation@sabi.ae; the system reads it, downloads the attached drawings and specs, classifies the discipline (electrical), runs an AI cable-take-off against the drawings, compares the result against market yardstick rates, generates a 12-section Power BOQ PDF, and — after the technical director's consent — emails the quotation back to the client.

Pipeline shape

The workflow has TWO levels:

- MAIN pipeline — 15 steps, 5 confirmation gates, runs from email arrival to quotation sent.
- ELECTRICAL sub-pipeline — 14 steps, runs INSIDE main step 11 (Run Pricing) on the Detailed path. It is the cable-take-off procedure: floor plans -> SLD -> cable schedule.

Two operating lanes

- Standard lane — every gate is a human checkpoint (George Varkey M, Technical Director, approves).
- INSTANT BOQ lane — the "Run to BOQ" button auto-approves Gates 1–4 and stops only at Gate 5 (Send to Client). Gate 5 is never collapsed; a human always sends the quotation.

Egress + cost guardrails

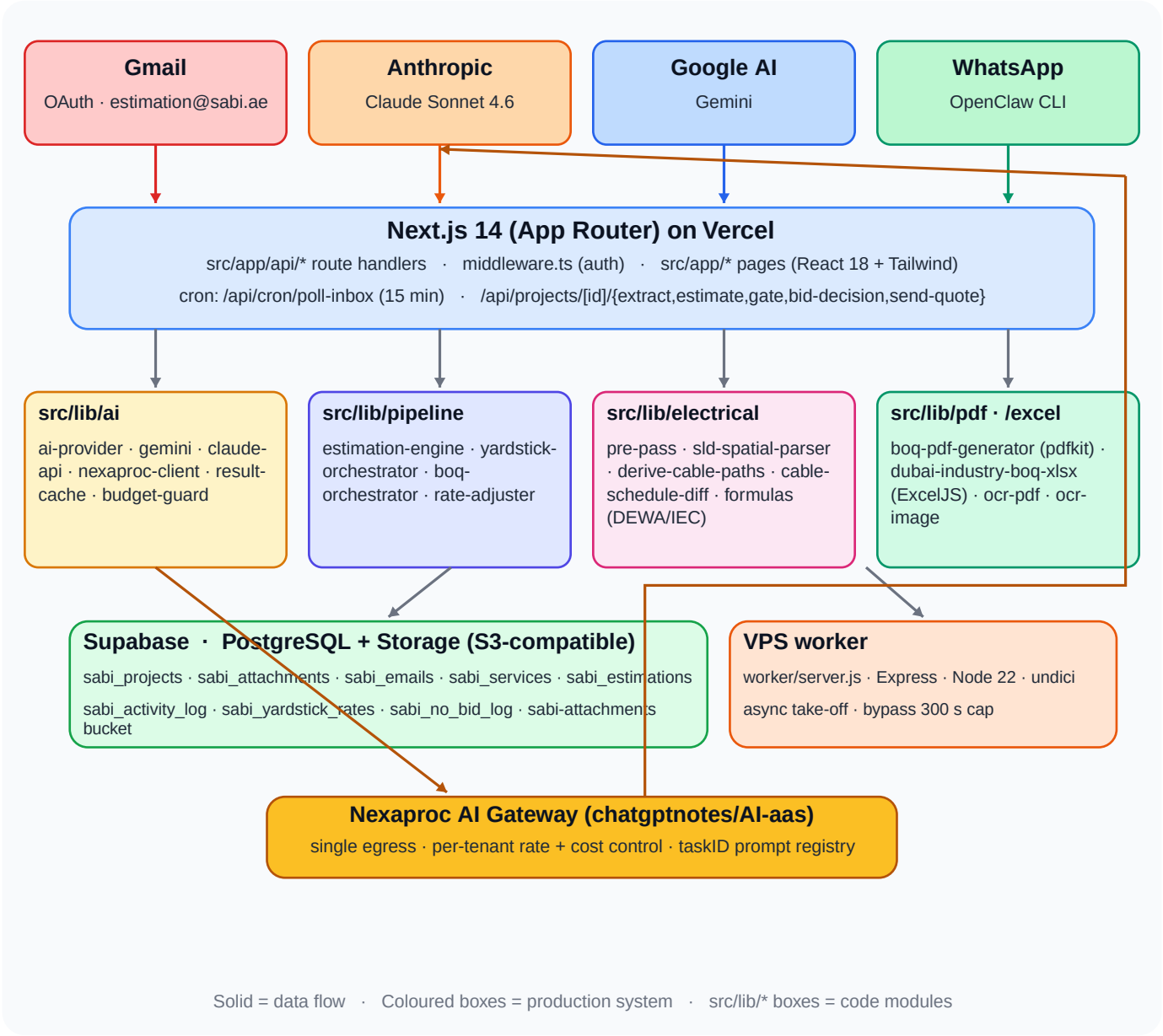
- Selective SELECTs (no SELECT *) on hot list pages to keep Supabase egress flat.
- AI calls go through the Nexaproc gateway with content-hash result caching (re-uploads of the same PDF do not re-bill Claude).
- Long take-offs (>300 s) are off-loaded to a VPS worker so Vercel's function cap never aborts a run.
- Per-error-kind throttled WhatsApp alerts (claude_401 / 429 / 529 / 5xx · gateway_timeout · cost_drift).

People & environments

- Approval authority: George Varkey M — Technical Director (george@sabi.ae).
- Source inbox: ESTIMATION_EMAIL (default estimation@sabi.ae) — only mail addressed here is treated as RFQ.
- Production: Vercel + Supabase (Postgres + S3-compatible Storage) + a Linode/VPS Express worker.

End-to-end architecture

Single-page mental model — every shape on this diagram maps to a folder in src/ or to an external system. Solid arrows are data; dashed arrows are control / events.



Solid = data flow · Coloured boxes = production system · src/lib/* boxes = code modules

How a single RFQ flows end-to-end

Each row is one event in time. The system column shows where the work happens.

#	What happens	Where (route / module)	DB / artifact
1	New email arrives at estimation@sabi.ae	Gmail	—
2	Cron polls Gmail every 15 min	Vercel cron · /api/cron/poll-inbox	sabi_emails…
3	Subject + body keyword pre-filter	lib/email/gmail-sync	RFQ_KEYWOR…
4	AI classification (priority + RFQ-or-not)	lib/ai/gemini.classifyEmail	sabi_projects…
5	Operator opens the bid detail page	app/bids/[id]/page.tsx	—
6	Run Extract — unzip + parse PDFs/specs	/api/projects/[id]/extract	sabi_attachmen…
7	AI building extraction (floors, area, type)	lib/ai/gemini.extractProjectInfo	sabi_projects.bu…
8	Discipline tag for every drawing	lib/ai/classifyDrawingDiscipline	sabi_attachmen…
9	Gate 1 — Documents Sufficient (binary)	/api/projects/[id]/gate	docs_sufficient_…
10	Gate 2 — Bid Decision (no_bid · detailed)	/api/projects/[id]/bid-decision	bid_decision_pe…
11	Run Pricing -> ENTERS electrical sub-pipeline	/api/projects/[id]/estimate	estimating
11.a	Pre-pass: SLD spatial · DXF text · OCR	lib/electrical/pre-pass	cache hash
11.b	Claude vision call (14-step prompt)	gateway -> claude CLI on VPS	ElectricalProced…
11.c	Long runs (>300 s) dispatched to VPS	worker/server.js	async write-back
11.d	Enrich result + diff against fixture	derive-cable-paths · cable-schedule-diff	cable_schedule…
12	Gate 3 — Confirm Quantities (=cable schedule)	/api/projects/[id]/gate	pricing_pending…
12.x	Auto-render 12-section Power BOQ PDF	pdf/boq-pdf-generator + pdfkit	storage://boq/{id…
13	Yardstick check (rates vs benchmarks)	pipeline/yardstick-orchestrator	yardstick_checked
14	Gate 4 — Confirm Total (AED + variance)	/api/projects/[id]/gate	consent_pending
15	Gate 5 — Consent -> Send (HUMAN)	/api/projects/[id]/gate	sending -> sent
x	Quotation email + PDF dispatched	lib/email/send-quotation	Gmail thread reply

Yellow rows = confirmation gates. Pink rows = electrical sub-pipeline (lives inside main step 11 on the Detailed path).

MAIN pipeline — 15 steps · 5 confirmation gates

Source: src/lib/shared/constants.ts -> MAIN_PIPELINE_STEPS. Status mapping in MAIN_STATUS_TO_STEP. Phases align with sabi-workflow.pdf v6.0.

1 Read Email

Poll inbox; pick mails to estimation@sabi.ae

status: classified

2 Register New Enquiry

Create sabi_projects row · project_name + source

status: enquiry_registered

3 Open Tender Folder

Create S3 prefix in sabi-attachments bucket

status: folder_opened

4 Unload Attachments

Save attachments · notify if none

status: attachment_unloaded

5 Extract Archive

Unzip / unrar · parse PDFs · OCR if needed

status: extracting

6 List Available Documents

Inventory + type tag (drawing/ spec/schedule)

status: documents_listed

7 List Drawings

Discipline tag every drawing (electrical only)

status: drawings_listed

8 Extract Building + Reputation

Floors, area/floor, building type, height · tier

status: building_extracted

9 Documents Sufficient G1

GATE 1 — binary: enough drawings/specs?

status: docs_sufficient_pending

10 Bid Decision G2

GATE 2 — 2-way: No-Bid · Detailed

status: bid_decision_pending

11 Run Pricing

Enters Electrical Sub (14 steps) on Detailed path

status: estimating

12 Confirm Quantities G3

GATE 3 — cable schedule review · auto-renders BOQ

status: pricing_pending

13 Yardstick Check

Compare AED against benchmark by building type

status: yardstick_checked

14 Confirm Total G4

GATE 4 — review total + yardstick variance

status: confirm_total_pending

15 Consent Received & Send G5

GATE 5 — HUMAN ONLY · dispatch quotation

status: consent_pending

Phase 1 · Information Sufficiency Phase 2 · Bid Decision Phase 3 · Quantities Phase 4 · Final Quote

Electrical sub-pipeline — 14 steps

Runs inside MAIN step 11 (Run Pricing) on the Detailed path. Source: ELECTRICAL_SUB_PIPELINE in src/lib/shared/constants.ts. Claude Sonnet 4.6 scans each drawing through a single 14-step prompt; the JSON result is enriched (lib/electrical/derive-cable-paths) and validated (cable-schedule-diff).

1 Open the Drawing
Locate every electrical drawing in the attachment set

2 List Available Drawings
Classify: floor_plan / schematic / riser / schedule / other

3 Establish Floors and Floor Height
Count + name every level · note typical floor height (m)

4 Find Drawing Scale
Read scale annotation or scale bar (1:100 / 1:50)

5 Identify LV Room / MDB
Find Main LV Panel · tag, rating (A), location

6 Check Schematic Drawing Availability
Confirm SLD / schematic exists · note filename

7 Note SMDBs from LV Panel
From SLD: tag · floor · rating · cable size (e.g. 4C×95mm²)

8 Identify SMDBs in Floor Drawings
Confirm SMDB locations Basement -> Roof

9 Establish Cable Route LV -> SMDBs
Read riser / annotations · note probable route

10 Estimate Cable Lengths LV -> SMDBs
mm² + length (m) + confidence: high/medium/low

11 Establish SMDB -> DB Identification
From SLD: every DB fed from each SMDB

12 Identify DB Locations per SMDB
From floor plans · floor by floor

13 Estimate Cable Size + Length per DB
Scaled floor-plan measurement · per-run confidence

14 GATE — Cable Schedule Review
Compile every cable entry · approve to render Power BOQ

On Gate 14 approval

1. status pricing_pending -> boq_generating
2. generateElectricalPowerBOQ() runs (lib/pdf/boq-pdf-generator.ts)
3. 12-section Power BOQ PDF stored at storage://sabi-attachments/boq/{id}/power-boq.pdf
4. status -> boq_ready · project re-enters MAIN step 13 (Yardstick)

AI provider routing & cost controls

src/lib/ai/ai-provider.ts is the router. Different functions go to different providers. All calls can be routed through the Nexaproc gateway by setting USE_AI_GATEWAY=true.

Per-function provider matrix

Function	Provider env	What it does
`classifyEmail`	AI_PROVIDER · Gemini	Email RFQ classification
`extractProjectInfo`	AI_PROVIDER · Gemini	Building info from spec PDF
`classifyDrawingDiscipline`	AI_PROVIDER · Gemini	Per-drawing discipline tag
`classifyReputation`	AI_PROVIDER · Gemini	Source-of-enquiry tier
`analyzeSpecifications`	AI_PROVIDER · Gemini	Spec doc requirements
`analyzeElectricalProcedure`	ELECTRICAL_AI_PROVIDER · Claude	14-step electrical take-off
`analyzeElectricalDrawing`	ELECTRICAL_AI_PROVIDER · Claude	Single-drawing electrical pass

Cost guardrails

- Result cache (lib/ai/result-cache.ts) — content-hash key on the input drawings; same PDF set hits cache, no Claude call.
- Fixture replay (lib/ai/test-fixture-replay.ts) — golden test fixtures replayed instead of live AI in test envs.
- Budget guard (lib/ai/budget-guard.ts) — hard ceiling per project · trips before runaway spend.
- Throttled WhatsApp alerts on claude_401 / 429 / 529 / 5xx · gateway_timeout · cost_drift · cohort_drift (lib/notifications/api-alert.ts).
- Daily cron /api/cron/ai-cost-drift watches per-project token spend versus rolling baseline.

Long-running take-offs (the >300 s problem)

Claude Opus on a 9.6 MB power PDF + the 14-step prompt regularly exceeds Vercel's 300 s function cap. The estimate route sets maxDuration=300 but, when worker-dispatch is enabled, off-loads to a VPS:

- POST -> /api/projects/[id]/estimate detects long-run conditions and calls dispatchEstimateToWorker().
- worker/server.js (Express, Node 22) calls the Nexaproc gateway with a 1200 s headersTimeout via undici dispatcher.
- Worker writes the ElectricalProcedureResult back to Supabase + sets status pricing_pending.
- The bid-detail page polls until the status changes — no UI change required.

Database & storage reference

Supabase (PostgreSQL). Migrations live in supabase/migrations/. Everything is project-scoped via project_id.

Table	Purpose
`sabi_emails`	Raw Gmail messages synced from estimation@sabi.ae
`sabi_email_attachments`	Attachment metadata at the email level (pre-project)
`sabi_projects`	One row per RFQ. Status drives the pipeline; bid_decision drives the path.
`sabi_attachments`	Attachment rows scoped to a project · file_type · discipline · storage_path
`sabi_services`	MEP services per project · confidence (high/medium/low) · pricing_source
`sabi_estimations`	Calculation results · margin · final_quote_aed · yardstick_status
`sabi_activity_log`	Audit trail per step · sub_pipeline column distinguishes MAIN vs electrical
`sabi_yardstick_rates`	Market benchmark rates by building type / discipline / region
`sabi_no_bid_log`	Terminal-exit audit · Gate 13 No-Bid + 7-day auto-escalation
`sabi_corrections`	Human corrections to AI output · feeds rate-adjuster + nb-tune
`sabi_settings`	KV store · gmail_sync_state, feature flags

Storage layout (sabi-attachments bucket)

- projects/{id}/raw/{filename} — original files extracted from email attachments
- projects/{id}/extracted/{filename} — files unzipped from archive attachments
- boq/{id}/power-boq.pdf — generated 12-section Power BOQ
- boq/{id}/power-boq.xlsx — Industry-format BOQ workbook (ExcelJS)
- preview/{id}/{att}.png — drawing thumbnails for the file viewer

File-format processing matrix

How each uploaded file is handled before the AI sees it.

Format	Pipeline	AI usage
PDF	pdf-parse -> text · pdfjs-dist -> page render · ocr-pdf if image-only	Sent to Claude/Gemini as vision
PNG / JPG	Buffer passed straight through · ocr-image when text needed	Vision input
DXF	dxf-parser server-side (lib/drawing/dxf-text-extractor.ts) · layer + TEXT/MTEXT	Text features feed AI as context
DWG	No native parser — REJECTED with operator hint	"Convert to PDF or DXF"
ZIP / RAR	adm-zip · node-unrar-js · entry-cap zip-bomb guard	Recurse into extracted files
XLSX / XLS	ExcelJS read · panel-schedule-parser · xlsx-schedule-parser	Structured rows feed AI
DOC / DOCX	mammoth -> text · spec-doc-loader	Spec text feeds AI
Images (TIFF/ BMP)	tesseract.js OCR fallback	OCR text feeds AI
EML / MSG	gmail.extractBody · stripQuotedReplies	Email text feeds classifier

Discipline detection — two passes

- Pass 1 (extract route): classifyDrawingDiscipline() scores filename + extracted text against keyword tables.
- Pass 2 (estimate route): re-classifies live · rejects files whose stored OR detected discipline is in NON_ELECTRICAL_DISCIPLINES.
- NON_ELECTRICAL_DISCIPLINES = { hvac, plumbing, fire_fighting, fire_alarm, bms, lpg, drainage }.
- Skipped files come back in the 422 response so the operator can see exactly why each file was excluded.

Power BOQ — 12 sections

- 1. Project Summary · 2. Incoming Supply & Transformers · 3. LV Panels
- 4. Sub-Main DB (SMDB) · 5. Distribution Boards (DB) · 6. Mechanical & Service Equipment
- 7. Power Outlets · 8. Cables — Main Distribution · 9. Containment
- 10. Earthing & LP · 11. Metering & Monitoring · 12. Summary of Electrical Loads

Source-tree map

Where to look when something needs changing.

Path	Purpose
src/app/api/cron/poll-inbox	Gmail polling cron — every 15 min
src/app/api/projects/route.ts	List projects (bid list)
src/app/api/projects/[id]	Project CRUD + sub-routes
extract/	Phase 1 extraction (steps 4–8)
estimate/	Phase 3 entry — runs electrical sub-pipeline
bid-decision/	Gate 2 — 2-way (no_bid · detailed)
gate/	Binary gates 9, 12, 14, 15
cable-schedule/	Cable schedule editor endpoints
yardstick/	Manual yardstick re-run
boq/ · power-boq/	BOQ Excel + Power BOQ PDF download
send-quote/	Email dispatch (Gate 5 trigger)
src/app/bids/[id]/page.tsx	Bid detail page (5056 lines · main UI)
src/app/inbox/page.tsx	Email inbox view
src/components/pipeline/	Workflow UI · sidebar, step timeline, modals
src/lib/shared/constants.ts	MAIN_PIPELINE_STEPS · ELECTRICAL_SUB_PIPELINE · gates
src/lib/ai/	Provider router · Gemini · Claude · gateway · cache
src/lib/electrical/	Pre-pass · SLD parser · cable derivation · DEWA formulas
src/lib/pipeline/	Estimation engine · yardstick · BOQ · rate adjuster
src/lib/pdf/	pdfkit setup · BOQ PDF generator · OCR
src/lib/excel/	Industry-format BOQ workbook (ExcelJS)
src/lib/drawing/	DXF text · scale detect · symbol counter · floor counter
src/lib/email/	Gmail OAuth · sync · send · personalization
src/lib/storage/	Supabase client · S3 multipart · activity logger
src/lib/notifications/	Throttled WhatsApp alerts (api-alert.ts)
supabase/migrations/	SQL schema migrations
worker/server.js	VPS background worker (Express + undici)
scripts/	CLI generators · seeders · fixtures

Glossary

- RFQ — Request For Quotation. The inbound email this system processes.
- BOQ — Bill of Quantities. The output document with line-item pricing.
- MDB / SMDB / DB — Main · Sub-main · Distribution boards (electrical hierarchy).
- SLD — Single-Line Diagram (the one-line schematic of the electrical system).
- Gate — A confirmation checkpoint. Binary (approve / revise) or 2-way (no-bid / detailed).
- Yardstick — Market benchmark rate. Used to sanity-check the AI estimate.
- INSTANT BOQ lane — auto-approve gates 1–4, stop at Gate 5. The "Run to BOQ" button.